

FitBuddy – AI Fitness Plan Generator using Gemini Models

Project Documentation

Project Description

FitBuddy is a web-based AI fitness planning application that generates a personalized 7-day workout plan and a practical nutrition or recovery suggestion based on user-provided information. The application is built using FastAPI and integrates Google Gemini models for AI generation.

The system accepts user information such as name, user ID, age, weight, fitness goal, and preferred workout intensity. It generates a structured workout plan, stores the user and plan information in SQLite using SQLAlchemy, and displays the result through a Jinja2-powered web interface.

FitBuddy also supports feedback-based plan updating. A user can submit feedback on an existing plan, after which the application uses the existing plan and feedback to generate an updated plan. An administrator can view stored users and plans through the protected All Users page.

Scenarios

Scenario 1: Personalized Workout Plan

A user enters their basic details, fitness goal, and preferred intensity. FitBuddy sends the information to the Gemini AI layer and generates a structured 7-day workout plan.

Scenario 2: Feedback-Based Plan Update

A user submits feedback on an existing plan. FitBuddy uses the original plan and the feedback to generate a revised workout plan and updates the stored result.

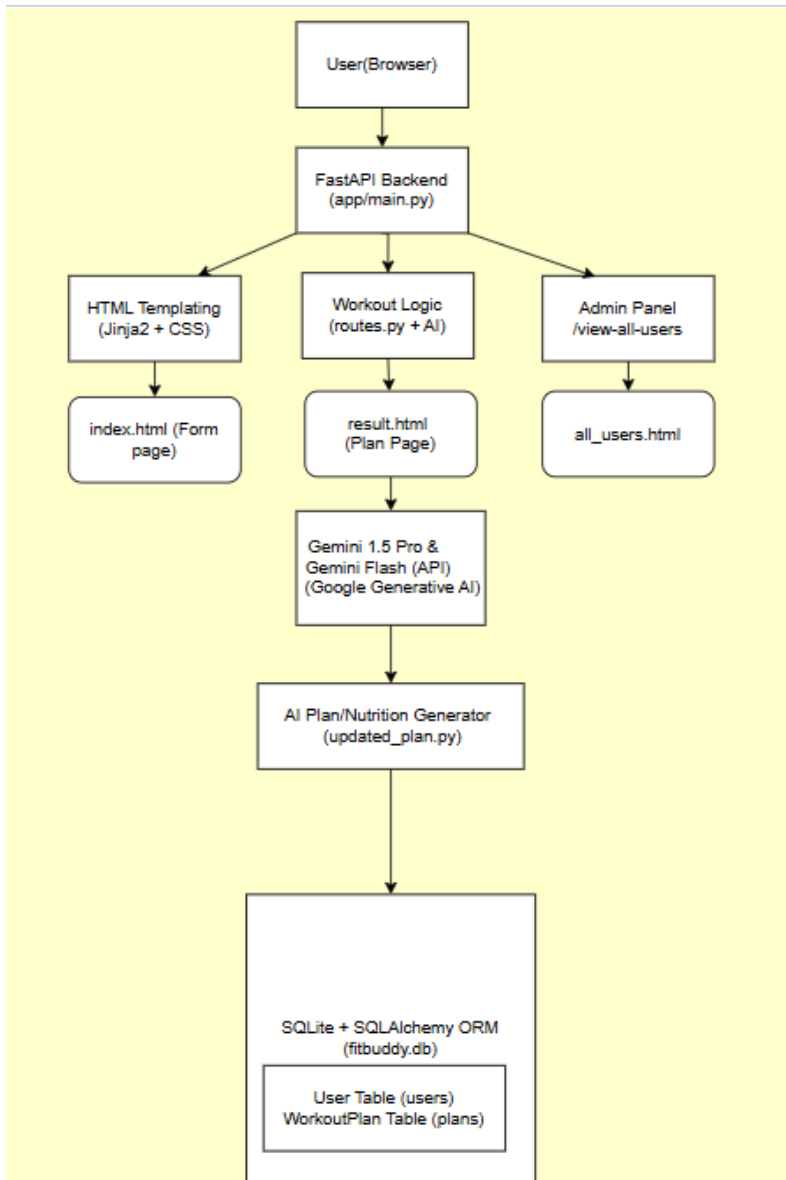
Scenario 3: Nutrition and Recovery Guidance

The system generates a concise nutrition or recovery suggestion related to the user's selected goal and displays it with the workout plan.

Scenario 4: Administrator User and Plan View

An authorized administrator accesses the protected All Users page to view registered users, their stored plans, updated plans, and feedback. The administrator can also delete a user and their associated plan.

Technical Architecture



Pre-requisites

- Python and pip
- FastAPI and Uvicorn
- HTML, CSS and Jinja2
- Python programming knowledge
- SQLAlchemy and SQLite basics
- Google Gemini API familiarity
- Git and GitHub for version control
- Virtual environment and environment-variable setup

Project Links

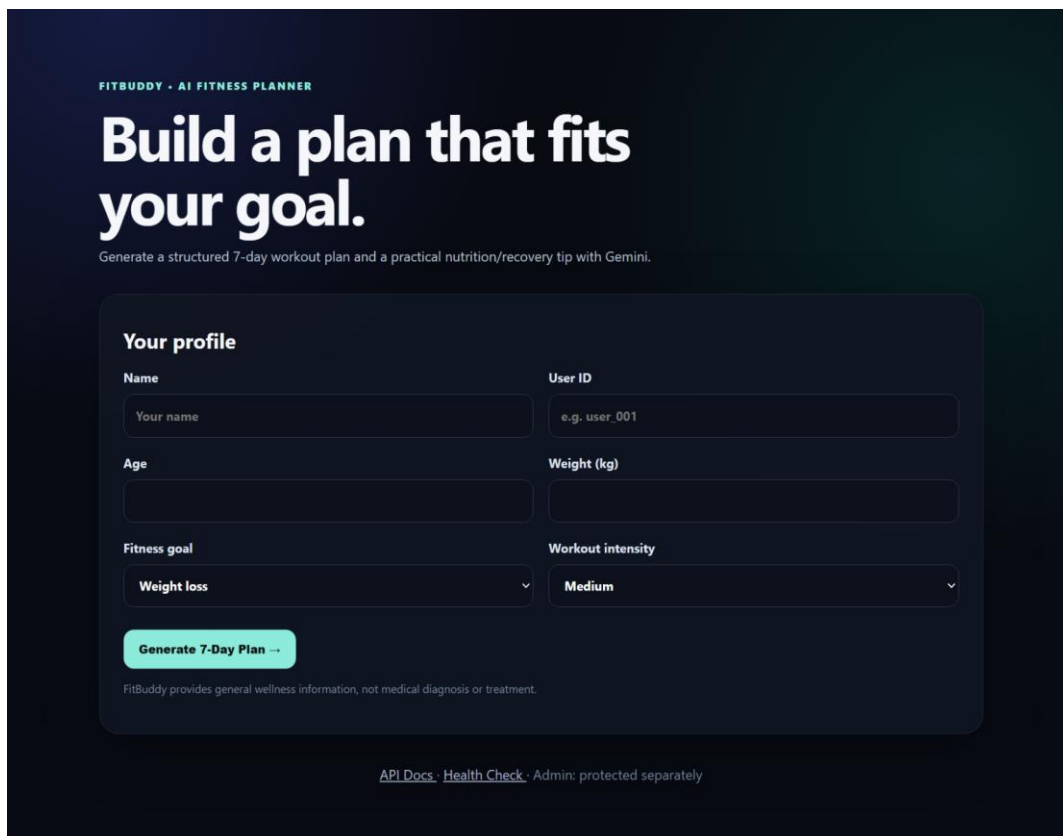
Local Application: <http://127.0.0.1:8000/>

FastAPI Documentation: <http://127.0.0.1:8000/docs>

Admin / All Users Page: <http://127.0.0.1:8000/view-all-users>

GitHub Repository: <https://github.com/rankawatdurga296-art/FitBuddy>

Demo Video: Use the Google Drive demo link submitted with the project.



The image shows a dark-themed web interface for 'FitBuddy - AI Fitness Planner'. At the top, the title 'Build a plan that fits your goal.' is displayed in large white font, with a subtitle 'Generate a structured 7-day workout plan and a practical nutrition/recovery tip with Gemini.' below it. The main form is titled 'Your profile' and contains several input fields: 'Name' (placeholder: 'Your name'), 'User ID' (placeholder: 'e.g. user_001'), 'Age', and 'Weight (kg)'. There are also two dropdown menus: 'Fitness goal' (currently set to 'Weight loss') and 'Workout intensity' (currently set to 'Medium'). A green button labeled 'Generate 7-Day Plan -->' is positioned below the form. At the bottom of the form, a small disclaimer states: 'FitBuddy provides general wellness information, not medical diagnosis or treatment.' Below the form, there are links for 'API Docs' and 'Health Check', followed by the text 'Admin: protected separately'.

FITBUDDY - AI FITNESS PLANNER

Build a plan that fits your goal.

Generate a structured 7-day workout plan and a practical nutrition/recovery tip with Gemini.

Your profile

Name
Your name

User ID
e.g. user_001

Age

Weight (kg)

Fitness goal
Weight loss

Workout intensity
Medium

Generate 7-Day Plan -->

FitBuddy provides general wellness information, not medical diagnosis or treatment.

[API Docs](#) · [Health Check](#) · Admin: protected separately

Project Workflow

- Milestone 1. Model Selection and Architecture
 - Activity 1.1. Research and Select the Appropriate Generative AI Model
 - Activity 1.2. Define the Architecture of the Application
 - Activity 1.3. Set Up the Development Environment
- Milestone 2. Core Functionalities Development
 - Activity 2.1. Develop the Core Functionalities
 - Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing
- Milestone 3. Route and Application Logic Development
 - Activity 3.1. Develop the Main Application Logic and API Routes
- Milestone 4. Frontend Development
 - Activity 4.1. Designing and Developing the User Interface
 - Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2
- Milestone 5. Deployment and Testing
 - Activity 5.1. Preparing the Application for Local Deployment
 - Activity 5.2. Testing and Verifying Local Deployment

Milestone 1. Model Selection and Architecture

This milestone focuses on selecting the generative AI model and defining the architecture required for FitBuddy. The application uses Gemini through an API-based integration with the FastAPI backend.

Activity 1.1: Research and Select the Appropriate Generative AI Model

FitBuddy requires an AI model capable of generating structured workout plans, concise nutrition or recovery guidance, and revised plans based on user feedback. The current implementation uses Gemini 3.1 Flash-Lite for these AI-generation tasks.

- Generate a structured 7-day workout plan.
- Generate a concise nutrition and recovery suggestion.
- Revise an existing workout plan using user feedback.
- Return structured responses that can be validated and displayed by the application.

Model Evaluation

We initially experimented with several Hugging Face models:

- T5 & BART – For text generation (workout/nutrition)
- DistilBERT – For potential intent classification
- Bloom – For general purpose generation

While these models showed promise in basic text generation, they had notable limitations.

- These models required:
- Hosting with Hugging Face or local GPU setup
- Fine-tuning or complex prompting to achieve structured plans

Final Model Selection

After comparative testing, we selected Google Gemini models for production:

- Gemini 1.5 Pro
 - Used for structured 7-day workout generation and plan updating
 - Delivers rich, context-aware, formatted responses reliably
- Gemini Flash
 - Used for generating fast, practical nutrition tips
 - Lightweight and efficient for quick API calls
- Reasons for Choosing Gemini:
 - API-based, easy to integrate with FastAPI
 - Naturally understands structured formatting (days, sections)
 - High accuracy with minimal prompt tuning
 - Scalable and fast for feedback-based revisions

Activity 1.2: Define the Architecture of the Application

- Frontend: HTML/CSS with Jinja2 templates.
- Backend: FastAPI routes and application logic.
- AI Integration: Google Gemini API.
- Database: SQLite with SQLAlchemy ORM.
- Server: Uvicorn.
- Security: HTTP Basic authentication for the administrator page.

The main user-facing templates are index.html and result.html. The project also contains all_users.html for the protected administrator view.

Frontend Responsibilities: Users interact via-

- index.html – Input form
- result.html – Plan, tip, feedback
- all_users.html – Admin panel

Backend Responsibilities:

- Receives form data
- Calls Gemini models
- Stores user info and plans
- Renders dynamic templates

AI Integration Points:

- Workout generation & feedback updates → Gemini Flash
- Nutrition tips → Gemini Flash

Activity 1.3: Set Up the Development Environment

The application is developed in Python using a virtual environment. Required packages are maintained in requirements.txt. Sensitive configuration such as the Gemini API key is stored in .env and excluded from GitHub using .gitignore.

The local application is started with:

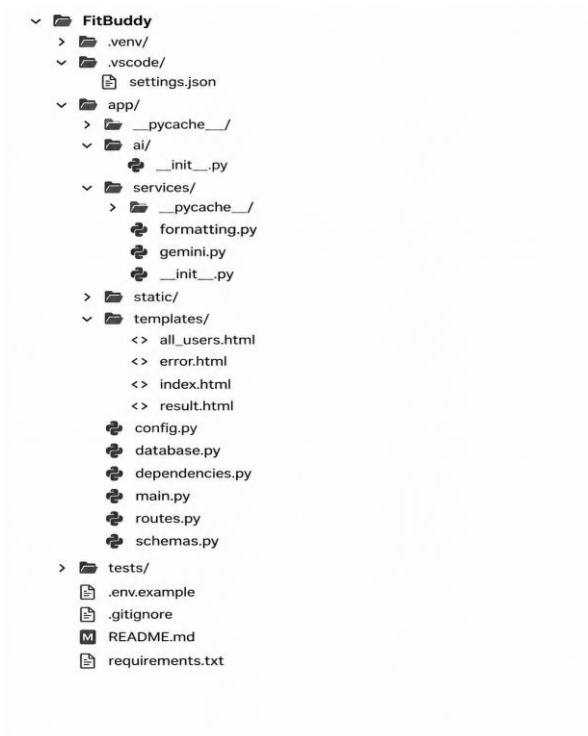
```
.venv\Scripts\Activate.ps1
```

```
uvicorn app.main:app --reload
```

The application is then accessed through <http://127.0.0.1:8000/> and FastAPI's interactive documentation is available at </docs>.

Set Up Project Structure:

Create folders: /app (Python files), /templates (HTML), /static/images (assets)



Milestone 2. Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

Workout Plan Generation

The `generate_workout_gemini()` function sends the user's profile and fitness preferences to Gemini and receives a structured `WorkoutPlan`. The application requires seven days and includes focus, warm-up, exercises, cooldown and safety guidance.

```
# Function to generate workout
def generate_workout_gemini(user_input):
    prompt = f"""
    You are a professional fitness trainer.

    Create a personalized, structured 7-day workout plan for someone with the goal of {user_input['goal']}, and prefers {user_input['intensity']}
    intensity workouts.

    Each day must include:
    - A warm-up (5-10 mins)
    - Main workout (targeted exercises, sets & reps)
    - Cooldown or recovery tip

    Format:
    Day 1:
    Warm-up: ...
    Main Workout: ...
    Cooldown: ...
    (Repeat for Day 2-7)
    """
    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error: {e}"
```

Nutrition and Recovery Generation

The `generate_nutrition_tip_with_flash()` function generates a concise practical nutrition or recovery suggestion. The result is stored with the workout plan and displayed on the result page.

```
def generate_nutrition_tip_with_flash(goal: str) -> str:
    """
    Generate a nutrition or recovery tip using Gemini Flash based on the user's fitness goal.

    Args:
        goal (str): User's fitness goal - "weight loss", "muscle gain", or "general fitness".

    Returns:
        str: Generated tip.
    """
    prompt = (
        f"Give one clear, helpful nutrition or recovery tip for someone focused on '{goal}'. "
        "The tip should be practical, friendly, and easy to understand."
    )

    try:
        response = model.generate_content(prompt)
        return response.text.strip()

    except Exception as e:
        return f"Error generating tip: {str(e)}"
```

Feedback-Based Plan Updating

The `update_workout_plan()` function receives the existing workout plan, user profile and feedback. Gemini then generates a revised 7-day plan while retaining useful parts of the original plan and applying the requested feedback safely.

```
def update_workout_plan(original_plan: str, user_feedback: str) -> str:
    """
    Use Gemini 1.5 Pro to update the workout plan based on user feedback.
    """
    prompt = f"""
    You are a professional fitness trainer assistant.

    Here's the original 7-day workout plan:
    {original_plan}

    User Feedback:
    "{user_feedback}"

    Based on the feedback, revise the relevant parts of the workout plan. Keep the format and rest of the plan unchanged if not needed.
    """
    try:
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Error updating plan: {e}"
```

User and Plan Storage

User information and generated plans are stored in SQLite through SQLAlchemy. The database stores user profile fields and plan information including the original plan, updated plan, feedback and nutrition/recovery information.

```

def save_user(user_id: int, name: str, age: int, weight: float, goal: str, intensity: str):
    db = SessionLocal()
    existing = db.query(User).filter_by(id=user_id).first()
    if existing:
        # Update existing user info
        existing.name = name
        existing.age = age
        existing.weight = weight
        existing.goal = goal
        existing.intensity = intensity
    else:
        # Create a new user
        user = User(
            id=user_id,
            name=name,
            age=age,
            weight=weight,
            goal=goal,
            intensity=intensity,
            schedule=7 # default schedule or logic
        )
        db.add(user)
    db.commit()
    db.close()

```

Activity 2.2: Implement the FastAPI Backend

- POST /api/generate-workout — generates and stores the workout and nutrition/recovery result.
- POST /api/submit-feedback — processes feedback and generates an updated plan.
- GET /api/users/{user_id} — retrieves stored user and plan information.
- GET /health — provides a simple application health response.
- GET /view-all-users — opens the protected administrator page.
- POST /admin/delete/{user_id} — deletes a selected user and associated stored data.

Milestone 3. Route and Application Logic Development

Activity 3.1: Main Application Logic and Routes

FastAPI routes connect the frontend forms, AI service functions and database operations. The routes validate incoming information, call the appropriate AI function, save results and return data or rendered templates.

The administrator functionality is implemented through the /view-all-users route. HTTP Basic authentication is checked before the route retrieves users from the database and renders all_users.html.

The administrator page displays each stored user's profile information, original plan, updated plan and latest feedback when available. The delete route removes a selected user and redirects back to the All Users page.

Set up Route Handlers for Each Feature

► / – Home Route

- Returns the homepage (index.html)
- No form processing logic

► /generate-workout – Plan Generator

- Receives user input using FastAPI's Form(...)
- Constructs a UserInput Pydantic model
- Calls:
 - generate_workout_gemini(...) → Generates workout plan via Gemini 1.5 Pro
 - generate_nutrition_tip_with_flash(...) → Gets nutrition tip via Gemini Flash
- Stores:
 - User details with save_user(...)
 - Plan with save_plan(...)
- Returns result.html with dynamic data

► /submit-feedback – Update Plan with Feedback

- Captures feedback and user_id
- Retrieves original plan from DB
- Uses update_workout_plan(...) to revise the plan via Gemini 1.5 Pro
- Updates the revised plan in the DB using update_plan(...)
- Renders updated result on result.html

► /view-all-users – Admin View

- Calls get_all_users() and get_all_plans() to fetch DB records
- Passes them to all_users.html
- Admin can view:
 - Username, Age, Weight, Goal, Intensity
 - Original and updated workout plans

Integrate Gemini AI Responses in Each Function

Each AI-related route calls Google's Gemini models with specific prompts:

- Workout Plan Generation:
generate_workout_gemini() sends fitness goal and intensity to Gemini Pro to return a structured 7-day

workout plan.

- Nutrition Tip Generation:
generate_nutrition_tip_with_flash() uses Gemini Flash to generate a practical tip related to the user's fitness goal.
- Workout Plan Update via Feedback:
update_workout_plan() passes original plan + user feedback to Gemini Pro and returns a revised plan.

All responses are returned to the user through result.html, maintaining clarity using <pre> blocks for formatting.

Workout generation route Nutrition/Recovery tip generation route

```
# 1. API: Generate workout using Gemini Pro
@router.post("/generate-workout/gemini")
async def generate_gemini_workout(request: WorkoutRequest):
    try:
        result = generate_workout_gemini({
            "goal": request.goal,
            "intensity": request.intensity
        })
        return {"model": "gemini-pro", "workout_plan": result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Nutrition/Recovery tip generation route

```
# 2. API: Generate nutrition tip using Gemini Flash
@router.get("/nutrition-tip")
def get_flash_tip(goal: str):
    tip = generate_nutrition_tip_with_flash(goal)
    return {"goal": goal, "nutrition_tip": tip}
```

Saving user info and plan generation route

```
# 3. API: Save user info & generate plan
@router.post("/generate-plan")
def generate_plan(user_data: UserInput):
    try:
        save_user(
            user_id=user_data.user_id,
            name=user_data.username,
            age=user_data.age,
            weight=user_data.weight,
            goal=user_data.goal,
            intensity=user_data.intensity
        )
        plan = generate_workout_gemini({
            "goal": user_data.goal,
            "intensity": user_data.intensity
        })
        save_plan(user_data.user_id, plan)
        return {
            "message": "Workout plan generated and saved successfully!",
            "workout_plan": plan
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Something went wrong: {str(e)}")
```

Submit Feedback and update plan route

```
# 4. API: Update workout plan based on user feedback
@router.post("/update-plan/{user_id}", response_model=dict)
def update_user_plan(user_id: int, data: FeedbackRequest):
    original = get_original_plan(user_id)
    if not original:
        return {"error": "Original plan not found for this user."}
    updated = update_workout_plan(original, data.feedback)
    update_plan(user_id, updated)
    return {"updated_plan": updated}
```

Database and Persistence

SQLite provides lightweight local persistence. SQLAlchemy ORM is used to query and update user and workout-plan records. The application can retrieve a stored user by User ID and can preserve the original plan, updated plan, feedback and nutrition/recovery information.

Milestone 4. Frontend Development

Activity 4.1: Designing and Developing the User Interface

The frontend provides a clean fitness-oriented interface for entering user information, viewing the generated plan, submitting feedback and accessing the administrator view when authorized.

- index.html — main user input page.

- result.html — personalized workout result, nutrition/recovery information and feedback form.
- all_users.html — protected administrator page for viewing stored users and plans.

The interface uses HTML and CSS styling with clear form fields, cards, buttons and plan sections.

Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

FastAPI's Jinja2Templates is used to render dynamic HTML pages. User information and generated results are passed from the backend to the templates. The result page displays the generated workout, nutrition/recovery information and feedback controls.

The All Users page uses Jinja2 loops and conditional blocks to display each stored user and the associated plan information.

```
from fastapi.templating import Jinja2Templates

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")

templates = Jinja2Templates(directory=TEMPLATE_DIR)
```

- Each route uses Template Response to render the proper HTML with passed context:

```
return templates.TemplateResponse("result.html", {
    "request": request,
    "username": username,
    "user_id": user_id,
    "age": age,
    "weight": weight,
    "goal": goal,
    "intensity": intensity,
    "workout_plan": plan,
    "nutrition_tip": nutrition_tip
})
```

- This allows dynamic plan and nutrition tip display after form submission.

Bind Backend Data to HTML Templates

► index.html

- Form elements use name="..." to bind to backend Form(...) parameters.
- Submits data to /generate-workout.

► result.html

- Uses:
 - {{ username }}, {{ goal }}, {{ intensity }}, etc. to show user data.

- {{ workout_plan }} and {{ nutrition_tip }} to show AI-generated results.
- Includes a feedback form submitted via POST to /submit-feedback.

► all_users.html

- Loops over all user records using:

```
</tr>
{% for user in users %}
<tr>
  <td>{{ user.id }}</td>
  <td>{{ user.name }}</td>
  <td>{{ user.age }}</td>
  <td>{{ user.weight }}</td>
  <td>{{ user.goal }}</td>
  <td>{{ user.intensity }}</td>
  <td><pre>{{ user.original_plan }}</pre></td>
  <td><pre>{{ user.updated_plan }}</pre></td>
</tr>
{% endfor %}
```

Milestone 5. Deployment and Testing

Activity 5.1: Preparing the Application for Local Deployment

Set up a virtual environment

- Create a virtual environment to manage dependencies and isolate the FitBuddy project from other environments.
- Activate the environment and install all necessary dependencies listed in the requirements.txt file, including FastAPI, Uvicorn, SQLAlchemy, and Google Generative AI SDK.

```
python -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Configure environment variables

- Use environment variables to store sensitive data such as the Gemini API Key.
- These variables can be set directly in the terminal or added to a .env file which is loaded into the application using python-dotenv.

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

Frontend-Backend Rendering via Jinja2

All user interactions are facilitated via Jinja2-rendered templates:

- index.html: Collects user inputs

- result.html: Displays the generated AI plan
- all_users.html: Lists all user records for admin view

The application is prepared for local deployment using a Python virtual environment, requirements.txt, environment variables and Uvicorn. The Gemini API key is stored in .env and is excluded from version control through .gitignore.

The project repository contains the source code and configuration needed to run the application. The local database file and secret environment file are not committed to GitHub.

Activity 5.2: Testing and Verifying Local Deployment

The application was tested locally by starting the FastAPI server and using the browser interface. The main workflow was verified from user input through AI generation, database storage, result display and feedback-based update.

Test Area	Expected Result	Status
Homepage	Input form loads successfully.	Verified
Workout generation	7-day structured workout is generated.	Verified
Nutrition/recovery	Tip and recovery note are generated.	Verified
Feedback update	Updated plan is generated from feedback.	Verified
Database	User and plan information is stored.	Verified
All Users	Authorized admin can view stored users and plans.	Verified
Admin delete	Selected user can be deleted.	Implemented
FastAPI /docs	Interactive API documentation loads.	Verified
Health endpoint	Returns application health status.	Implemented

Access the Application Locally

The FastAPI development server must be started using the command (uvicorn main:app --reload). Once the server is running, the application can be accessed through any web browser by navigating to <http://127.0.0.1:8000>. This URL loads the main user interface where individuals can input their details and generate personalized fitness plans. Additionally, the interactive API documentation provided by FastAPI can be viewed at <http://127.0.0.1:8000/docs>, allowing developers to explore and test available endpoints.

Exploring Website's web pages:

Home page:

FITBUDDY - AI FITNESS PLANNER

Build a plan that fits your goal.

Generate a structured 7-day workout plan and a practical nutrition/recovery tip with Gemini.

Your profile

Name
Your name

User ID
e.g. user_001

Age

Weight (kg)

Fitness goal
Weight loss

Workout intensity
Medium

Generate 7-Day Plan -->

FitBuddy provides general wellness information, not medical diagnosis or treatment.

[API Docs](#) · [Health Check](#) · Admin: protected separately

Description: The homepage of **FitBuddy – AI Workout Generator** features a clean and engaging user interface designed to provide a seamless experience for users seeking personalized fitness plans. The form prominently displayed in the center allows users to enter essential information including:

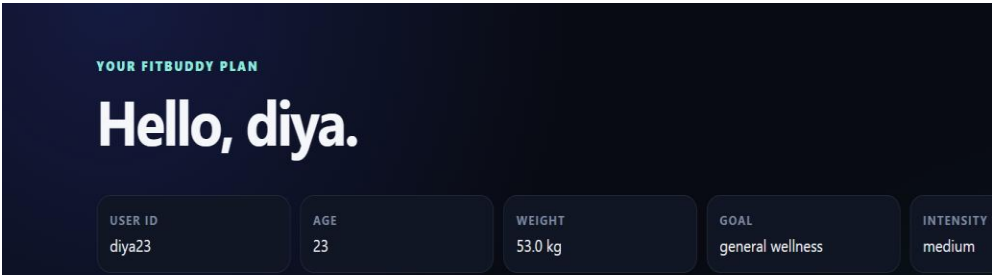
- **Name**
- **User ID**
- **Age**
- **Weight (kg)**
- **Fitness Goal** (e.g., weight loss, flexibility)
- **Workout Intensity** (Low, Medium, High)

Once users submit this data, they can click the **"Generate Plan"** button, which triggers the AI-powered backend to generate a customized workout and nutrition plan tailored to their goals and fitness profile. The visual appeal of the background image featuring an active individual reinforces the fitness-centric theme of the application, motivating users to take charge of their health journey.

This intuitive design ensures users of all experience levels can quickly get started with minimal effort.

Personalized workout generation page:

User Information:



Description: Once the user submits their details, the application navigates to a personalized workout plan page. This section displays a summary of the user’s input including their **Name**, **User ID**, **Age**, **Weight**, **Fitness Goal**, and **Selected Intensity**. This personalized display ensures users feel acknowledged and guides the AI engine to generate relevant workout recommendations tailored specifically to their fitness journey.

Workout Plan:

7-DAY WORKOUT

Original plan

Diya's Balanced Wellness Plan

MONDAY

Focus: Full Body Mobility and Strength

Warm-up: 5 minutes of light walking and arm circles

Exercises:

1. Bodyweight Squats – 3; 12 repetitions; Rest: 60 seconds
2. Incline Push-ups – 3; 10 repetitions; Rest: 60 seconds

Cooldown: 5 minutes of gentle static stretching for legs and chest

Safety: Ensure your movements are controlled and stop immediately if you feel sharp pain.

TUESDAY

Focus: Cardiovascular Health

Warm-up: 3 minutes of marching in place

Exercises:

1. Brisk Walking – 1; 30 minutes; Rest: N/A

Cooldown: 3 minutes of slow walking and deep breathing

Safety: Maintain a pace where you can hold a conversation comfortably.

WEDNESDAY

Focus: Active Recovery

Warm-up: Gentle neck and shoulder rolls

Exercises:

1. Yoga Flow or Stretching – 1; 20 minutes; Rest: N/A

Cooldown: 5 minutes of seated meditation

Safety: Listen to your body; if a stretch feels uncomfortable, ease out of it.

THURSDAY

Focus: Core and Stability

Warm-up: 4 minutes of light jogging in place

Exercises:

1. Bird-Dog – 3; 10 repetitions per side; Rest: 45 seconds
2. Plank – 3; 30 seconds; Rest: 60 seconds

Cooldown: 5 minutes of child's pose and cat-cow stretches

Safety: Keep your back neutral and avoid arching during core movements.

FRIDAY

Focus: Functional Movement

Warm-up: 5 minutes of dynamic movements like side-to-side steps

Exercises:

1. Step-ups onto a sturdy surface – 3; 10 repetitions per leg; Rest: 60 seconds
2. Wall Sits – 3; 30 seconds; Rest: 60 seconds

Cooldown: 5 minutes of full-body stretching

Safety: Ensure your equipment is stable before stepping on it.

SATURDAY

Focus: Recreational Activity

Warm-up: Light movement relevant to your chosen activity

Exercises:

1. Leisurely bike ride, hike, or dance – 1; 45 minutes; Rest: N/A

Cooldown: 5 minutes of gentle walking

Safety: Stay hydrated and choose an environment that feels safe and enjoyable.

SUNDAY

Focus: Rest and Restoration

Warm-up: None required

Exercises:

1. Rest Day – 1; Full day of rest; Rest: N/A

Cooldown: None required

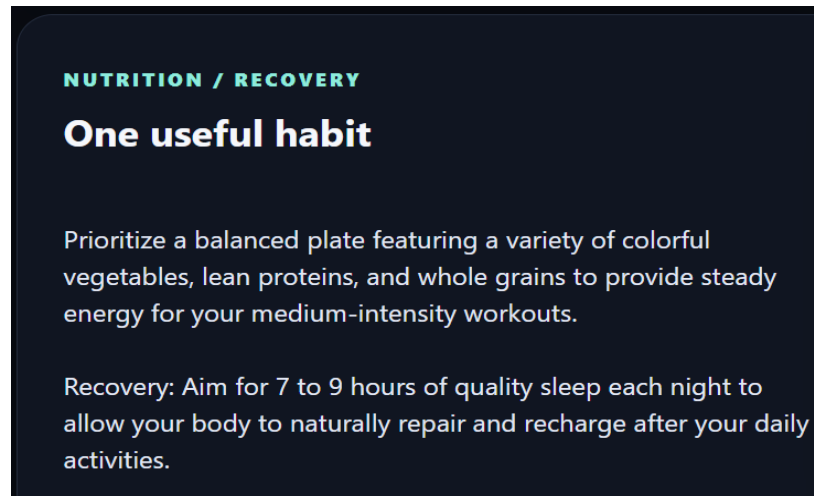
Safety: Rest is essential for recovery; prioritize sleep and relaxation today.

GENERAL NOTES

- This plan is adaptable; feel free to swap days based on your schedule.
- Always prioritize proper form over speed or intensity.
- If you experience persistent pain or discomfort, consult with a healthcare professional.
- Focus on how your body feels during and after movement rather than specific outcomes.

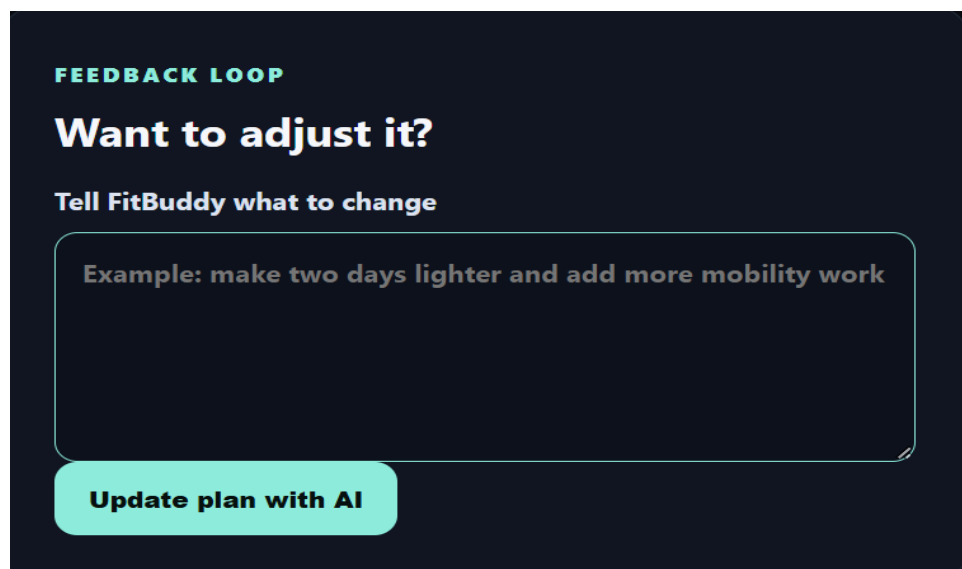
Description: FitBuddy generates a personalized, day-wise workout plan tailored to the user's fitness goal, experience level, and preferred workout schedule. The workout section on the result page outlines each day with a clear focus, such as full body, upper or lower body, cardio, core strength, or flexibility. For every workout day, a curated list of exercises is provided, detailing the name of the exercise, the number of sets and repetitions (or duration), and suggested rest intervals. The plan incorporates variety and structured progression to keep the workouts engaging and effective. Additionally, basic warm-up and cool-down guidance may be included to support safe and well-rounded training. Overall, this section aims to deliver a goal-oriented, easy-to-follow workout schedule that promotes consistency and progress.

Nutrition Tip:



Description: The **Nutrition Tip** section in FitBuddy provides users with concise, practical dietary advice to complement their workout plans. It emphasizes the importance of prioritizing protein intake in every meal, suggesting sources like chicken, fish, beans, or Greek yogurt. The tip explains how protein contributes to muscle building, satiety, and metabolic support—key factors for both fat loss and muscle gain. This section is designed to reinforce healthy eating habits that align with users' fitness goals.

Feedback Page:



Description: The Feedback Page in FitBuddy allows users to personalize and refine their fitness journey by submitting feedback on their current workout plans. Users are prompted to enter their unique User ID and provide specific feedback regarding the effectiveness, difficulty, or preferences related to their plan. Once submitted, this feedback is processed by the system to intelligently update and adjust the workout plan to better align with the user's goals, comfort, and progress. This dynamic feedback loop ensures that each fitness plan remains adaptive and tailored to the individual's evolving needs.

Once the plan is updated it comes with a confirmation message .



View all users page:

FITBUDDY ADMIN

Users & plans

Home

kana #kana7

Delete

22 years · 55.0 kg · muscle gain · low

Original plan

Kana's Gentle Muscle Gain Plan

MONDAY

Focus: Full Body Strength

Warm-up: 5 minutes of light walking and arm circles

Exercises:

1. Dumbbell Goblet Squats – 3; 10-12 reps; Rest: 90 seconds

Note: Use a light weight that feels comfortable.

2. Dumbbell Overhead Press – 3; 10-12 reps; Rest: 90 seconds

Note: Keep core engaged and back straight.

Cooldown: 5 minutes of gentle stretching

Safety: Stop immediately if you feel sharp pain.

TUESDAY

Focus: Active Recovery

Warm-up: Light walking

Exercises:

1. Leisurely Walk – 1; 20 minutes; Rest: N/A

rohini #rohi67

Delete

26 years · 69.0 kg · weight loss · medium

Original plan

Rohini's Balanced Weekly Wellness Plan

MONDAY

Focus: Full Body Strength

Warm-up: 5 minutes of light walking and arm circles

Exercises:

1. Bodyweight Squats – 3; 12 reps; Rest: 60 seconds

2. Incline Push-ups – 3; 10 reps; Rest: 60 seconds

Cooldown: 5 minutes of gentle static stretching for legs and chest

Safety: Listen to your body and stop if you feel any sharp pain.

TUESDAY

Focus: Cardiovascular Health

Warm-up: 3 minutes of marching in place

Exercises:

1. Brisk Walking – 1; 30 minutes; Rest: N/A

Cooldown: 3 minutes of slow walking and deep breathing

Safety: Ensure you are wearing supportive footwear.

Description: The View All Users page in FitBuddy provides an overview of the entire user database, displaying detailed information for each registered user. This includes their personal details such as name, age, and fitness goals, along with their original workout plan generated at registration and the updated workout plan if any changes were made based on their submitted feedback. This feature enables a transparent view of how each user's plan has evolved over time, showcasing the adaptability of the FitBuddy system and helping administrators or developers monitor user progress and engagement effectively.

Conclusion

FitBuddy demonstrates an AI-powered fitness planning workflow using FastAPI, Jinja2, SQLite, SQLAlchemy and Google Gemini. The application accepts user information, generates a structured 7-day workout plan, provides nutrition and recovery guidance, stores the result and supports feedback-based plan updates.

The project also includes a protected administrator view for reviewing stored users and plans. The application was tested locally through the browser and FastAPI's interactive documentation. The architecture is modular and can be extended in the future with additional features such as authentication improvements, deployment to a cloud platform, richer analytics and further personalization.

Future Improvements

- Deploy the application to a cloud hosting platform.
- Add stronger administrator authentication and role management.
- Improve database reporting and plan history.
- Add richer progress tracking and analytics.
- Improve responsive design for different screen sizes.
- Add additional AI-assisted wellness planning features while maintaining safety-focused guidance.

Project Repository and Demo

GitHub Repository: <https://github.com/rankawatdurga296-art/FitBuddy>

Local Application: <http://127.0.0.1:8000/>

FastAPI Documentation: <http://127.0.0.1:8000/docs>

Demo Video: Use the Google Drive share link already prepared for the project submission.